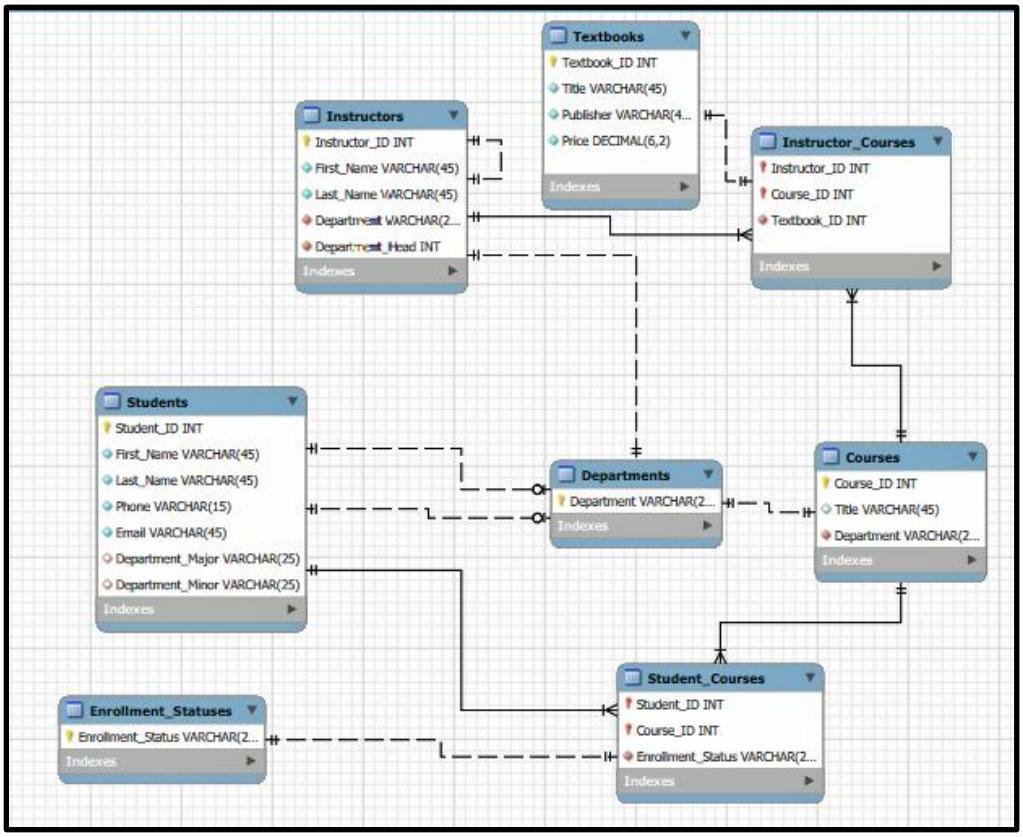
Chapter 19A: SQL

In this chapter we are going to take a look at SQL, the Structured Query Language. SQL is used to manipulate data in a database. There are two main flavors of SQL: DDL the Data Definition Language and DML the Data Manipulation Language.

DDL is used primarily by database designers and administrators to create schemas, define tables, manage performance and establish data quality criteria. DML is used by programmers to insert, update and delete data.

We will start with DDL example. Let's jump to the inside and take a look at the E-R diagram we will be using for this example. (Figure 19A.1)

Figure 19A.1: E-R Diagram



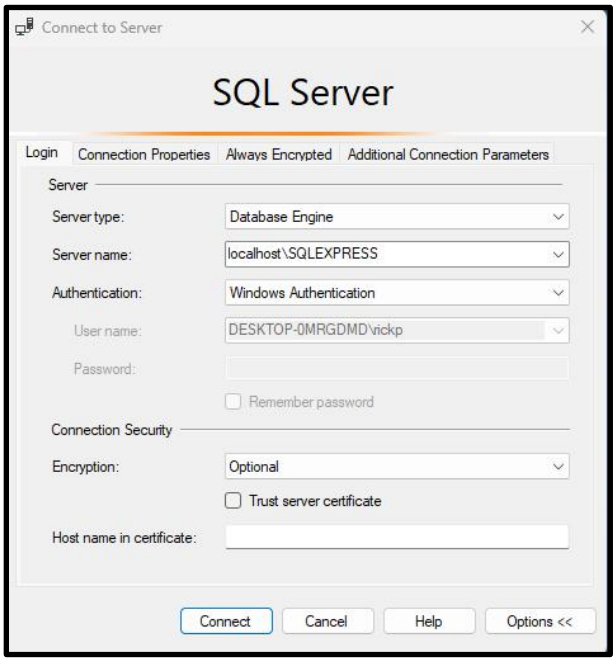
Let's take a look at this ER and talk about what it says and how you might discuss this diagram with a group of subject matter experts.

- There are 5 real-world entities depicted in the diagram: Instructors, Textbooks, Courses, Students and Departments.
- Students may have a declared major and minor department. If these declarations exist, they must be valid department names. Note, these fields are nullable, so it is possible that a student has not yet declared a major and/or minor.
- The subject matter experts tell you valid enrollment statuses include: Major, Minor, Elective and Audit. Each course enrollment by a student must indicate one of these enrollment statuses.
- Each course must be associated with a valid department name.
- Each instructor is associated with a valid department name, and each department has an instructor who also serves as the department head.
- Instructors may teach zero, one or many courses and each course may be taught by zero, one or many instructors. Each instructor-course combination is associated with a valid textbook.
- Each student may enroll in zero, one or many courses and each course may host zero, one or many students. Each student-course combination is associated with a valid enroll status value.

Let's assume that our subject matter experts agree with our representation of their world and that we are ready to proceed to defining our schema. By the way, if I inadvertently refer to a schema as a database please forgive me. Everybody who works with big data uses the two terms interchangeably.

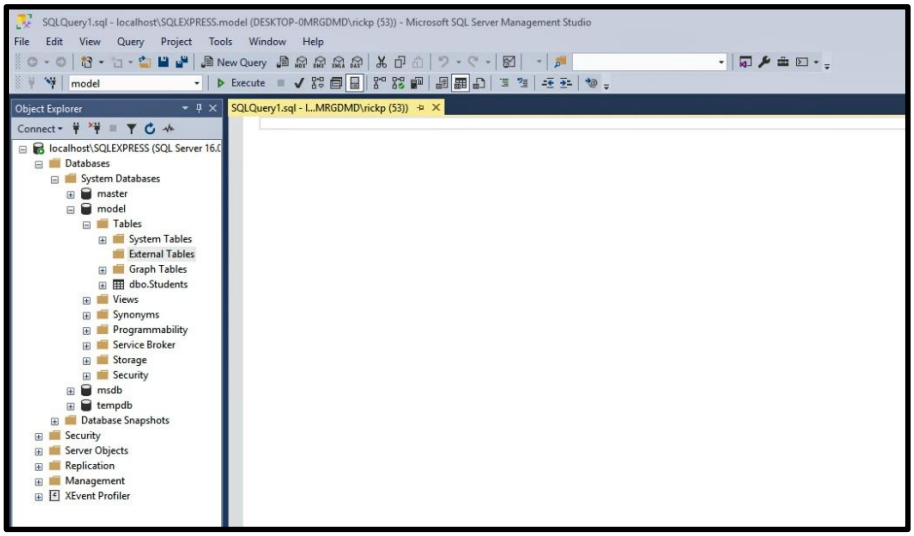
We begin by opening SQL Server Management Studio. The default connection information shown here should work for you, but if it doesn't please go back to that information you wrote down when we first installed SQL Server. (Figure 19A.2)

Figure 19A.2: SQL Server Connection via SQL Server Management Studio



Once connected please click the “New Query” icon in the top menu. Your screen should now look like this. (Figure 19A.3)

Figure 19A.3: New Query Screen



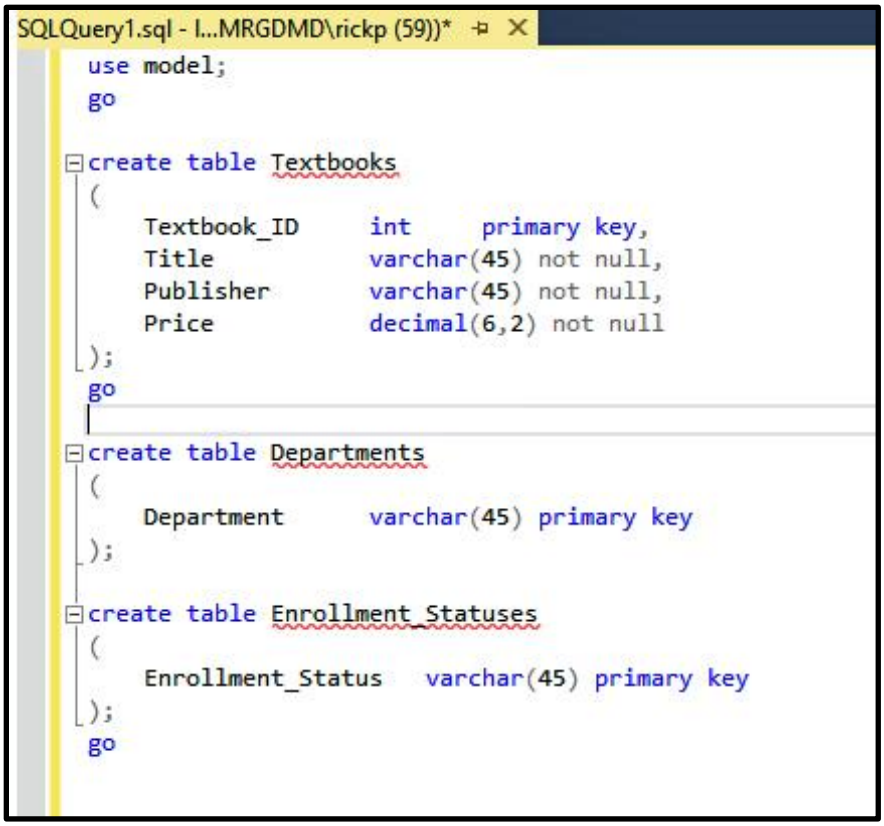
We will begin by defining the Textbooks table, but first we have to tell SQL Server which schema we are going to use. We do this by typing *use Model;* and then on a separate line the keyword *go*. The keyword go delimits a set of executable commands. To execute this command simply click the execute button at the top of the your screen.

The create table syntax is shown here. (Figure 19A.4) The create table keywords are self-explanatory. Next comes the table name followed by a parentheses enclosed list of field names and their associated data types. You may also optionally include the keywords primary key and/or not null. In our case the Textbook_ID field is the primary key of this table and none of the fields are nullable. The Departments and Enrollment_Statuses tables utilize a similar syntax.

By the way, field names and keywords in SQL are not case sensitive. Table names are not case sensitive in SQL Server or Oracle by default. White space is allowed anywhere in SQL to improve readability.

To execute this SQL DDL (Data Definition Language) simply select the to-be-executed commands and click the execute icon.

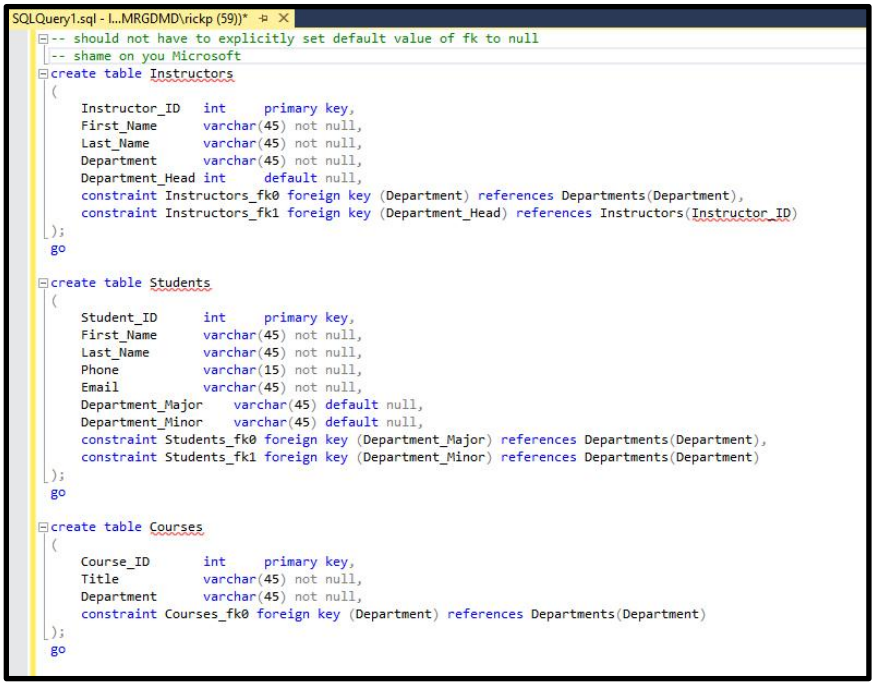
Figure 19A.4: Create Table



We next create the Instructors, Students and Courses tables. (Figure 19A.5) Please note how I had to set a default value of null for some of the foreign key fields. This is not ANSI standard SQL DDL and Microsoft is throwing you a bug here. By default, all foreign key fields should be nullable. It took me until I tried to start entering data into these fields that I discovered the problem. The drop table and alter table commands quickly become your friends when dealing

with new versions of SQL Server and Oracle. Both come complete with undocumented features, a.k.a. bugs.

Figure 19A.5: Foreign Keys



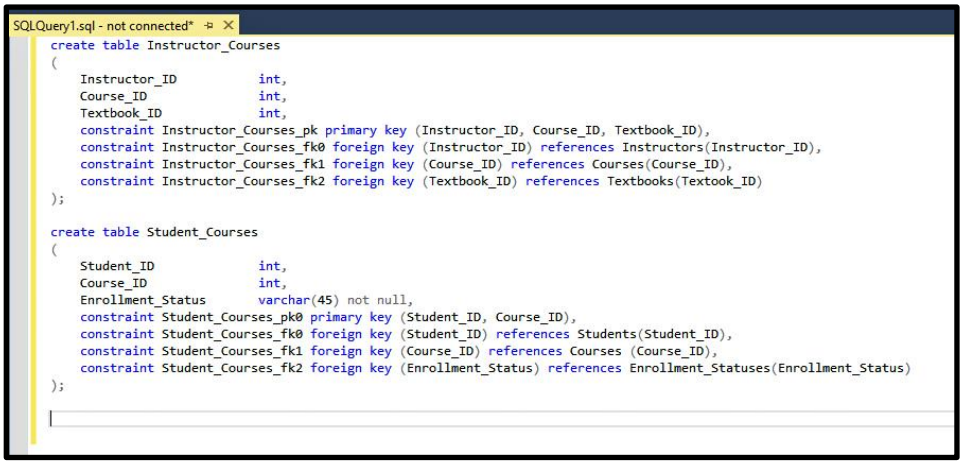
```
SQLQuery1.sql - I...MRGDMD\rickp (59))*
-- should not have to explicitly set default value of fk to null
-- shame on you Microsoft
create table Instructors
(
    Instructor_ID   int     primary key,
    First_Name      varchar(45) not null,
    Last_Name       varchar(45) not null,
    Department      varchar(45) not null,
    Department_Head int     default null,
    constraint Instructors_fk0 foreign key (Department) references Departments(Department),
    constraint Instructors_fk1 foreign key (Department_Head) references Instructors(Instructor_ID)
);
go

create table Students
(
    Student_ID      int     primary key,
    First_Name      varchar(45) not null,
    Last_Name       varchar(45) not null,
    Phone           varchar(15) not null,
    Email           varchar(45) not null,
    Department_Major    varchar(45) default null,
    Department_Minor    varchar(45) default null,
    constraint Students_fk0 foreign key (Department_Major) references Departments(Department),
    constraint Students_fk1 foreign key (Department_Minor) references Departments(Department)
);
go

create table Courses
(
    Course_ID       int     primary key,
    Title           varchar(45) not null,
    Department      varchar(45) not null,
    constraint Courses_fk0 foreign key (Department) references Departments(Department)
);
go
```

We culminate our DDL with the Instructor_Courses and Student_Courses tables each of which has a compound primary key and multiple foreign key references. Please note that compound primary keys must be created using the constraint syntax. (Figure 19.6)

Figure 19A.6: Compound Primary Keys



```
SQLQuery1.sql - not connected*
create table Instructor_Courses
(
    Instructor_ID           int,
    Course_ID               int,
    Textbook_ID             int,
    constraint Instructor_Courses_pk primary key (Instructor_ID, Course_ID, Textbook_ID),
    constraint Instructor_Courses_fk0 foreign key (Instructor_ID) references Instructors(Instructor_ID),
    constraint Instructor_Courses_fk1 foreign key (Course_ID) references Courses(Course_ID),
    constraint Instructor_Courses_fk2 foreign key (Textbook_ID) references Textbooks(Textook_ID)
);

create table Student_Courses
(
    Student_ID              int,
    Course_ID               int,
    Enrollment_Status       varchar(45) not null,
    constraint Student_Courses_pk0 primary key (Student_ID, Course_ID),
    constraint Student_Courses_fk0 foreign key (Student_ID) references Students(Student_ID),
    constraint Student_Courses_fk1 foreign key (Course_ID) references Courses (Course_ID),
    constraint Student_Courses_fk2 foreign key (Enrollment_Status) references Enrollment_Statuses(Enrollment_Status)
);
```

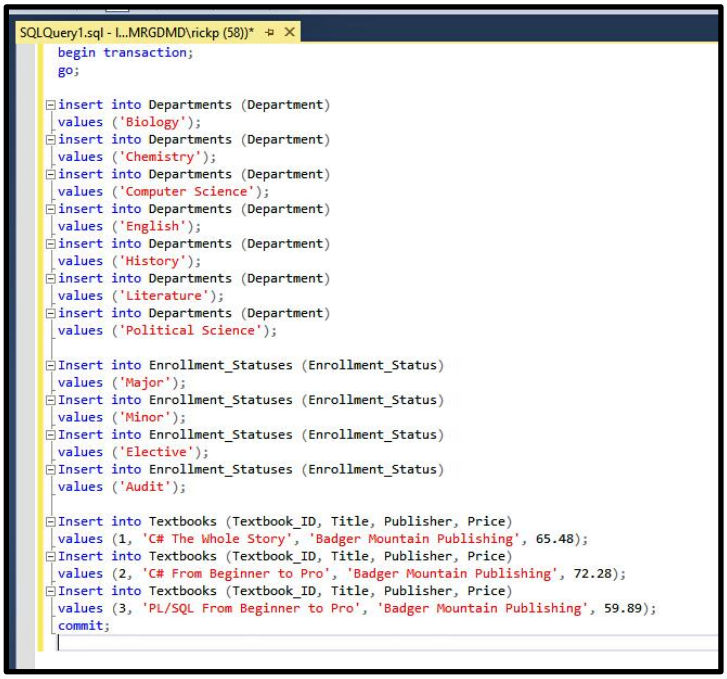
We now have all our tables and relationships defined so we can leave the DDL world and move onto DML, the Data Manipulation Language.

We begin with some insert commands on the Departments table. (Figure 19A.7) The syntax is very straightforward. You begin with the keywords insert into and then the table name. The table name is followed by a parentheses enclosed list of the fields to be populated. Next comes the keyword values followed by a parentheses enclosed list of the to-be-inserted values. Note, the order here of the fields and values is important. Field number one is going to get value number one and so on. If the data types don't match it will throw an error. The problem really arises when the data types do match, but you still have the order of the values messed up. You end up sticking values into fields you did not intend.

The syntax for the Textbooks and Enrollment_Statuses tables follows the same pattern. The only thing to note is the *begin transaction;* and *commit;* syntax. The begin transaction syntax says this is the beginning of a bunch of insert, update or delete commands that all must be completed together. The commit command says if you get this far without an error go ahead and put these updates to the database. If an error has occurred anywhere throughout the begin transaction – commit sequence, rollback all the updates such that the database looks like nothing ever occurred.

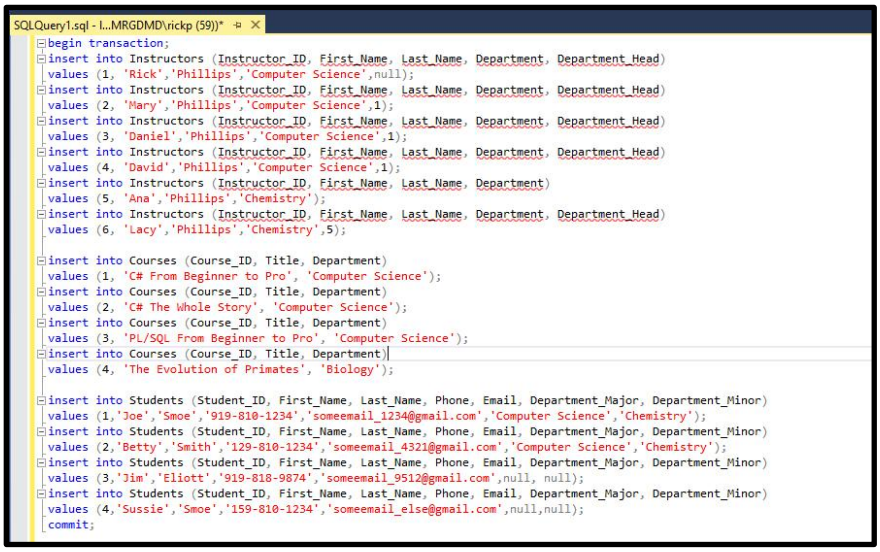
This phenomenon is known as transaction processing. In our case it is no big deal but what if you were posting financial transactions? I bet you want all the debits to post with their corresponding credits or if something goes wrong throw the whole mess out and start again.

Figure 19A.7: Insert



Next, it was time to deal with the Instructors, Courses and Students tables. Please note how I have two different ways to deal with nullable fields. You can either specify the field name in the original list and then enter the keyword null as its value or you can simply omit the field name and fall back on its default value, i.e. null. (Figure 19A.8)

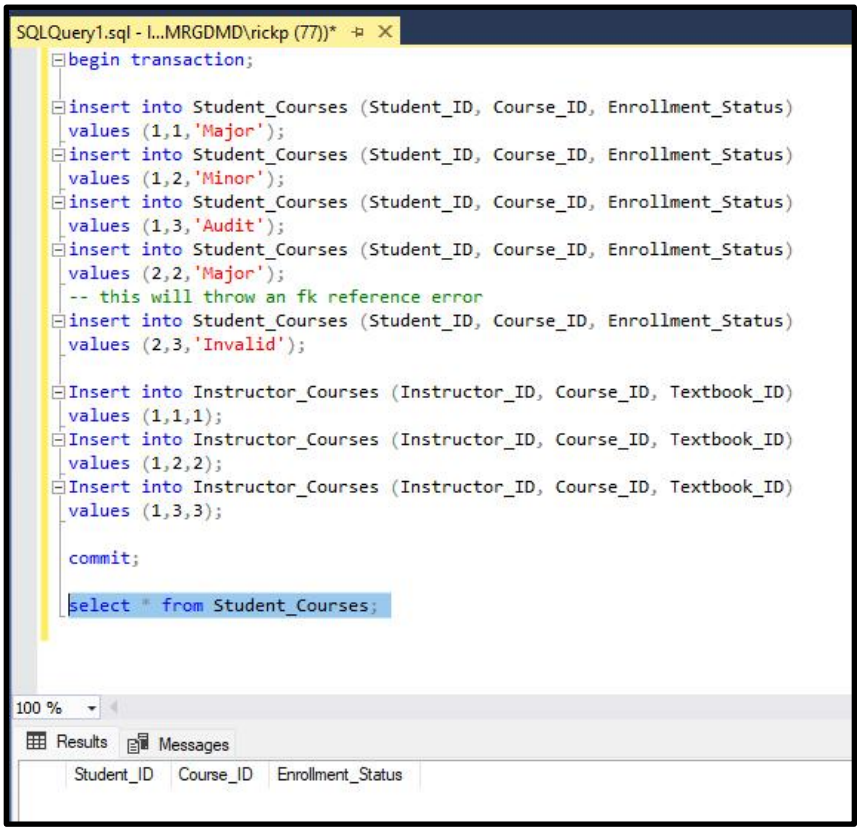
Figure 19A.8: Nullable Field Options



I next try to populate the Student_Courses and Instructor_Courses tables while simultaneously throwing a foreign key violation into the middle of the transaction. (Figure 19A.9)

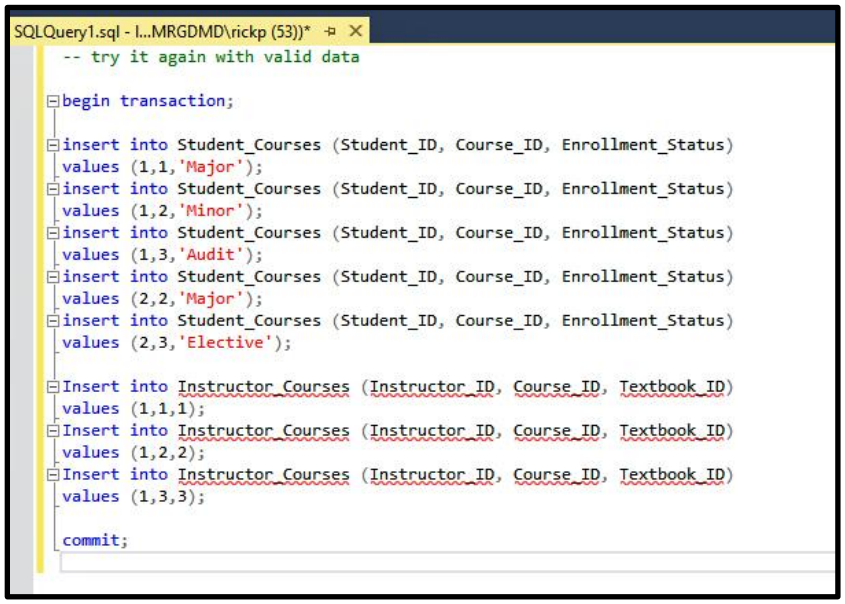
First off, SQL Server told me I had thrown an fk error when I executed this set of commands but those statements are not visible here because I wanted to show you the results of this select command. This select command simply says, “show me all the columns and rows in the Student_Courses table.” Check it out. Because I had one error in this transaction string nothing happened in any of the commands, i.e. the entire transaction was rolledback. Guys, this is going to be huge when you start dealing with transactions made up of multiple steps.

Figure 19A.9: Transaction Rollback



Now let’s try this set of commands without the fk reference error. (Figure 19A.10) Okay, that looks better.

Figure 19A.10: Insert to Compound PK



Okay, that about puts a wrap on creating a schema and inserting some data. In our next lesson we will learn how to update, delete and select information from a schema. Should be great fun.